

AGENTIC AI

超级智能体系统架构峰会

Data Agent 生态

我们的上半年

被质疑

怎么做

做到了哪

冯浩哲 · 腾讯TEG大数据智能体算法负责人

WeData

腾讯云 BI

全民 BI

AMS Adata

 **DataFun.**

今年我们遇到的最大的问题，不是技术



Claude Code 这么强，把你们的接口开放成 MCP 和 Skills，让它直接干活不就行了——你们为什么还要做一个 Data Agent?

这半年，这句话我们被反复问。今天诚实地讲讲我们怎么回答。

01

质疑阶段

2026.01 - 2026.03

CLI harness 爆火，「干掉中台」成为最有感染力的图景

- 以 Claude Code 为代表的 CLI harness 爆火
- 人手一个 Agent CLI，目标：干掉 SaaS，去掉中台
- 中台只需开放 Skills / MCP，本地接上 Agent 就开干

双线作战 · 其一

把 Skills / MCP 做好，适配各家 CLI harness

双线作战 · 其二

重新回答——**我们为什么有必要存在？**

为此，我们对 Claude Code（约 **57 万行** TypeScript）做了逐模块的深度拆解。结论：三个技术原因 + 一个场外原因。

原因一 · 代码是文本，数据不是

	代码	数据
本质	纯文本	结构化记录，一个分区动辄 10GB
怎么看	grep / glob / read，直接进上下文	放不进上下文，只能靠 SQL 间接访问
成本	毫秒级 · 本地磁盘 I/O	分钟级 · 集群计算资源
策略	错了再搜，反复渐进	必须一次找准

Claude Code 没做任何代码索引——纯文本搜索快到成本可以忽略。但你没法对一张几亿行的表做 grep。

原因二 · 代码随时有正反馈，数据没有

代码 · 反馈回路闭合 ✓

写代码 → 编译 / 运行 → 报错 = 明确反馈 → 模型自我修正 → 在 1M 上下文里反复试错

🕒 反馈回到模型，循环往复——这是 Code Agent 成立的根基

数据 · 反馈回路断开 ✗

写 SQL → 平稳运行 · 零报错 → 条件不对 / join 写错 / 选错字段 / 理解错口径 → 南辕北辙的数字 ✗

唯一的「反馈」：用户自发纠正，或默默不再使用

数据场景最阴险的地方：失败是静默的。没有任何正反馈能让模型做对。

原因三 · 缺乏中心化维护的 Skills，一定会腐化

95% → 65%

Anthropic 官方博客实测：精心维护的 Skills 下准确率 95%；用户增加、维护减少，一个月后跌至 65%

沉淀团队经验，**中心化是必然的抉择**

次要原因（一带而过）

- Code Agent 指令空间被占满：9 个代码工具 + system prompt $\approx$ 1 万 token，模型能严格遵循指令的有效窗口业界经验仅 ~16K
- Skills listing 预算仅上下文 1%、每条描述限 250 字符；MCP 工具全部延迟加载
- → 在 Coding Agent 里，data 永远是二等公民

场外原因

安全——数据接入外部 CLI 后，审计链路中断。这也在推着我们做中心化 Data Agent。

02

到底怎么做：三个痛点，三件事

一一对应，没有一件是多余的

数据不是文本



统一库表知识体系

数据没有正反馈



SQL + Python 混合计算

Skills 会腐化



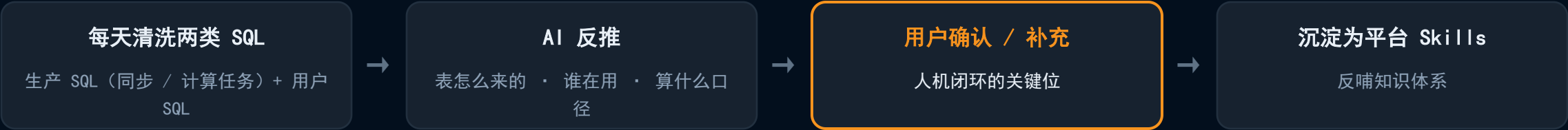
LLM as a Judge 知识运营

事一 · 统一库表知识体系

对标 Unity Catalog · 解决「检索不动」



规模挑战：日活跃库表千万级、字段上亿，comment 大面积空白——纯人工梳理走不通 → 于是有了 AI 增强知识系统：



用 AI 把执行代码洗成业务知识

41% → 79%

找表召回率 · 视频号业务数百条真实 Query 实测 · 已逼近人工精标上界

事二 · SQL + Python 混合计算

解决「没有正反馈」

业界主流 · SQL 端到端派

Databricks + Metric View, 语义层收口径

我们的现实

数据源横跨 hive / PG / StarRocks / MySQL, 没有统一方言与引擎;
且复杂 SQL 生成准确率大幅下跌

SQL 按死在 ETL 取数 · 下推数仓

只许: 选分区 / 选字段 / 选条件 / 基础聚合

(min · max · sum · avg · count · distinct) / 仅 left join

大数据过滤聚合在数仓完成, TB 级也算得动

大数据进



小结果集出

分布式 Python 接手分析

百 GB 级内存计算——特征工程 / 建模 / 可视化

🕒 中间结果、报错

直接回给模型自我修正

数据分析, 终于也有了「编译器」

56% → 90%

腾讯云 BI 真实业务问题集准确率

事三 · LLM as a Judge 评测系统

解决「Skills 会腐化」

Anthropic 的教训——知识建好只是开始，不运营就是慢性死亡（95% → 65%）

每晚 · LLM 自动评测

对当天每一条用户 tracing 打分，分流
两类 case

✓ 满意 / 纠错后完成任务

萃取用户知识 → 写成 Skills → 反哺知识体系——用户每一次纠错 = 平台一次进化

✗ 明确负面情绪 （「不是这样」「你懂不懂啊」「继续继续」）

收进 badcase 池 → 挨个研究怎么解

痛点一
知识体系解决找不到

痛点二
混合计算解决没反馈

痛点三
评测运营解决会腐化

03 · 我们的路线 vs 行业

2026.6 Data + AI Summit 给出的场外答案

Databricks：同样被问「要不要把平台能力 MCP / CLI 化」——答案：优先中心化平台

- 真正以「自研 Agent 调 MCP servers」为主流用法的，只有极少数具备成熟自研能力的头部客户；绝大多数客户不具备去中心化运营能力
- DBX 的判断：「平台一旦被工具化，可替代性就大幅提升」
- 押注 Genie One + Genie Ontology 自动知识图谱——准确率大幅跃升，显著优于标准 RAG



Snowflake · Horizon Context

「没有上下文，在猜；有上下文，agent 才干活」

Anthropic · 内部实践

准确率 21% → 95%

OpenAI · 内部实践

复杂问题回答 22 分钟 → 1 分 22 秒

决定 Data Agent 上限的，是中心化维护的上下文知识层——正是我们年初顶着质疑选的路

诚实的观察

Databricks 也不轻松——真正把 Genie One 用好的客户仍是少数，单客户交付动辄以月计、重度依赖专家驻场。方向共识已形成，坑大家都在填。这是我们的机会窗口。

成绩单 · 学术 + 商业双轨验证

SPIDER 2.0-LITE · 免训练赛道

60.15 / 基线 23 分

免训练 Agent 全球 #2 · 国内 #1

Agent 框架 ×2.6 增益 · 客户任意标准模型可复现

BIRD · 训练派赛道

75.01 全球 #4

国内云厂商唯一 Top 10

前三: Google 77.14 / AWS 76.47 / Databricks 75.68 · 距 Google 仅 2.13 分

真实客户盲测 · 能源 / 零售 / 制造三家腾讯云客户

83% 数十个真实 case

与 Databricks Genie 并列 #1

领先国内头部竞品 10pp 以上 · 前三名我们占两席

内部业务实测

85% - 92%

多场景稳定在线

金融问数 · 通用问数 · 广告代码生成等多个内部场景

学术榜跑得快 · 外部客户打得过 · 内部业务用得上——三件事一起立住，这条路才算走通

回到年初那个问题

数据不是文本，所以我们做了**知识体系**

数据没有正反馈，所以我们做了**混合计算**

知识会腐化，所以我们做了**中心化运营**

这半年，这三件事各拿到了自己的成绩单；Databricks 和 Snowflake，也在今年走上了同一条路。

检索 / 知识体系

四层知识与渐进式披露如何做到 F1 领先 SOTA

计算

DSL→SQL 下推 + Python 沙箱突破内存墙

评测 / 知识运营

LLM as a Judge 如何让知识保鲜

 **DataFun.**

AGENTIC AI

超级智能系统架构峰会

Data Agent设计实战： 知识准备、库表检索与SQL-Python混合计算

演讲嘉宾：王柳焮

腾讯TEG大数据Data Agent负责人

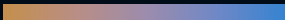
目录

CONTENT

- 01 数据链路的Agent ic重构
- 02 知识准备
- 03 库表检索
- 04 SQL-Python混合计算
- 05 总结与展望

01

数据链路的Agentic重构



切入真实数据链路



“上个月各业务线流水对比情况如何？”——用户只问了一句话



1 库表查询

找哪张表、哪些字段
能回答这个问题

→ 流水明细表、渠道维度表
`amt, channel, dt`

2 代码生成

用 SQL / Python
写出计算逻辑

```
SELECT channel,  
       SUM(amt)  
GROUP BY channel
```

3 数据计算

在引擎上执行
处理规模与安全

→ 分布式引擎执行
数据不出域

4 结果生成

解释结果
形成可读结论

→ 对比图表 + 文字
“A 业务线环比 +18%”

用户只问了一个业务问题，背后却是四个环节依次衔接。

Data Agent 设计实战的本质，就是对这条数据链路的**重构**：搞清楚每一环节 Agent 要做什么、会遇到什么问题、该如何解决，又该如何把全链路串联起来。

聚焦典型业务痛点

一次看似简单的业务提问，会在找表、对口径、做计算三个环节连续受阻。

01 找表难

元数据缺失严重

表和字段语义大量空白，元数据缺失率约 85%。

人工治理不可持续

头部业务动辄上千张表、每表数百字段，依靠人工标注难以覆盖。

02 口径乱

业务语义存在多重歧义

“上个月流水”同时牵动组织、指标与时间三类定义。

理解业务才是关键

SQL 语法之外，还需要准确识别企业内部的指标定义与业务语义。

03 算不动

本地计算存在内存瓶颈

pandas 面对大规模数据容易撞上内存墙，TB 级任务常需工程师改写 PySpark。

数据搬运带来风险

数据下载到本地会拉长分析周期，并增加安全与合规成本。

提出三条主线解法

围绕数据分析链路的三个断点，分别建立可被 Agent 自主调用的**知识、检索与计算能力**。

01

知识准备

解决“Agent **看不懂数据**”的问题

将无语义、分散的数据资产沉淀为可理解、可检索、可更新的业务知识。

02

库表检索

解决“Agent **找不到正确数据**”的问题

在运行时按需定位正确的表、字段和口径，并逐步消解业务歧义。

03

SQL-Python 混合计算

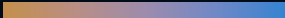
解决“Agent **面对大数据算不动**”的问题

在可信环境中结合 SQL 下推、Python 沙箱与分布式异构计算完成分析闭环。

02

知识准备

🔗 解决“Agent 看不懂数据”的问题。



业务语义标准化

知识准备

先将数据资产沉淀为 Agent 能理解、能检索、能执行 的标准知识

统一语义层

AGENT 标准数据语料库

01

指标标准化

定义、粒度与计算逻辑保持一致

02

AI 可信查询

业务词典、系统过滤与 Golden Queries

03

多源数据虚拟化与整合

跨引擎形成统一的数据视图

04

自主决策

Agent在治理规则下执行操作，形成从洞察到行动的闭环

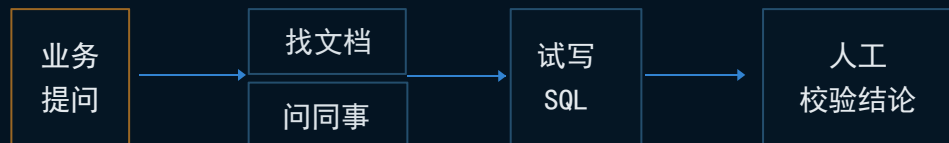
以统一语义层承接业务经验，让每一次数据分析都对齐到**同一套定义与口径**

面临的问题

01 元数据治理难

元数据治理：数月

一次业务提问的常见路径

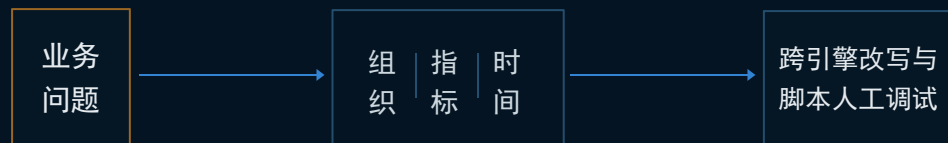


- ◆ **元数据质量低**：表和字段标注为空的比例约 **85%**，语义依赖人工口口相传。
- ◆ **数据孤岛**：采集、加工与分析分散在不同部门，数据知识难以融合，检索链路不完整。
- ◆ **规模化阻碍**：头部业务动辄上千张表、每表数百字段，人工标注消耗大量人力。

02 多种计算口径

交付周期：周级

同一业务问题的工程化路径



- ◆ **跨引擎方言**：同一查询在 **SR / PG / Spark / MySQL / SuperSQL** 等引擎间存在语法差异。
- ◆ **复杂代码门槛高**：脚本需反复跑通、读报错、修改，研发生效慢，复用成本高。
- ◆ **口径歧义**：“上个月流水”同时牵动组织、指标、时间三类定义，结果难以稳定对齐。

关键举措

用 AI 将生产 SQL、查询行为、Schema、数据分布和血缘逻辑，沉淀为 Agent 可消费的数据语料库，使数据资产具备可理解的业务语义。

01 自动元数据增强

生产 SQL

真实业务查询与计算逻辑

查询流水

表与字段的使用行为

原始 Schema

结构、类型与命名信息

库级业务背景

业务域、主题与场景解释

LLM 逆向推导表语义

从真实数据使用信号中还原表在业务链路中的位置与价值。

- ◆ 这张表从哪里来
- ◆ 这张表被谁使用
- ◆ 这张表解决什么业务问题

02 字段级 Profiling 增强

为何需要 Profiling

字段名和 comment 经常不足以表达业务含义；数据分布能够揭示字段是维度、指标、时间或实体 ID。

业务价值

自动识别字段角色，判断字段能否支撑问题，并减少语义缺失带来的召回偏差。

取值分布

直方图

最大 / 最小
/ 均值

枚举 Top-N

码值字典

字段角色

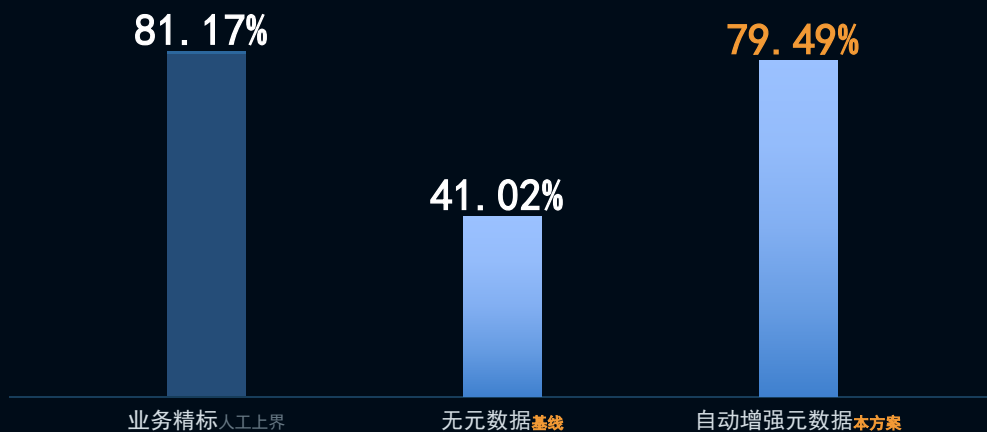
直接收益

01 自动元数据增强

基于数据血缘 + 执行代码，解决“无语义”问题，提升数据资产可理解性

视频号找库表准确率

439 Query 测试集

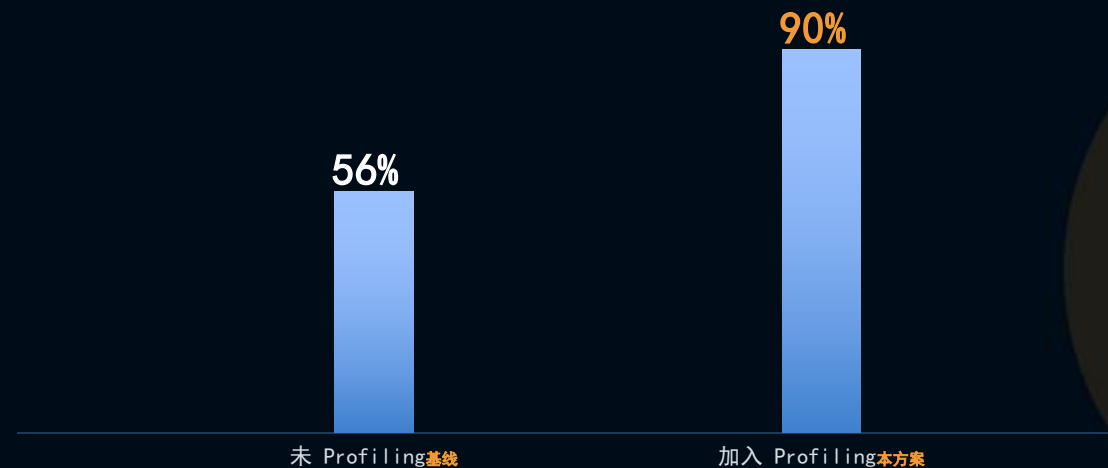


02 Profiling 增强效果

用数据特征反推业务含义，补齐业务/数据的“隐性知识”

腾讯云 BI 协同 · 问数准确率

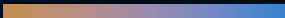
数值越高越准



03

库表检索

🔗 解决“Agent 找不到正确数据”的问题。



从人工检索到Agent检索

知识准备沉淀“有哪些可用知识”；运行时检索回答“当前问题该用哪些知识”。

01

经验驱动

- A 翻 Wiki、问同事、试写 SQL
业务知识分散在文档、系统和个人经验中。
- B 问题可能走向不同候选表
检索路径缺少稳定过程，结果难以复用与审计。
- C 人工反复验证业务口径
组织、指标、时间与过滤条件需要项项核对。

检索效率差异

02

运行时按需调用

- A 从业务问题定位候选表
先用轻量级知识缩小范围，避免全量 schema 进入上下文。
- B 用字段与数据分布验证可用性
确认指标、维度和过滤条件是否由候选表支撑。
- C 按需追溯血缘与计算逻辑
将“相似表”收敛为可解释、可执行的正确口径。

渐进式披露检索

L1-L4 分层知识体系



AGENTIC AI
超级智能系统架构峰会

分层检索设计 将数据知识从轻到重拆开，让 Agent 只获取当前判断所需的最小信息。

L1 表级元数据

快速定位候选业务表

表名 / 中文名
摘要 / 业务主题
数据粒度

回答：哪些表可能相关？

L2 字段元数据

校验指标与维度承载能力

字段名 / 数据类型
字段描述 / 角色
主键与关联字段

回答：字段能否支撑问题？

L3 字段 Profiling

用真实数据补齐字段语义

取值分布 / 枚举 Top-N
码值字典
最大 / 最小 / 均值

回答：字段语义是否可信？

L4 血缘与计算逻辑

解决复杂口径与跨表关系

上下游关系 / 字段血缘
派生公式 / 指标逻辑
Join Path

回答：口径是否可以追溯？

轻量信息 · 快速
缩小范围

按需下钻 · 随信息需求
逐层加载

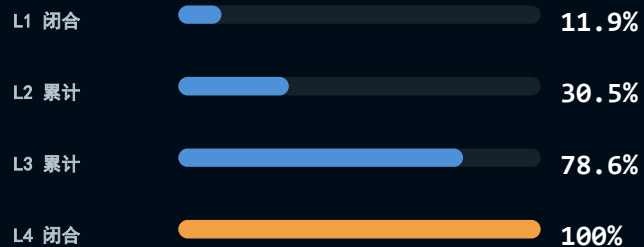
复杂知识 · 精确
对齐口径

检索过程中候选表逐步收敛到正确口径，上下文随不确定性的消解逐层补充，让每次调用都服务于当前判断。

落地成果

检索闭合率

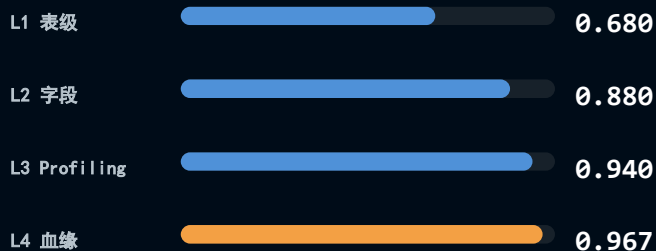
Spider 2.0 · 547 case



L3 独立贡献 48.1%，字段真实取值与码值信息是闭合率提升的关键。

累计召回率

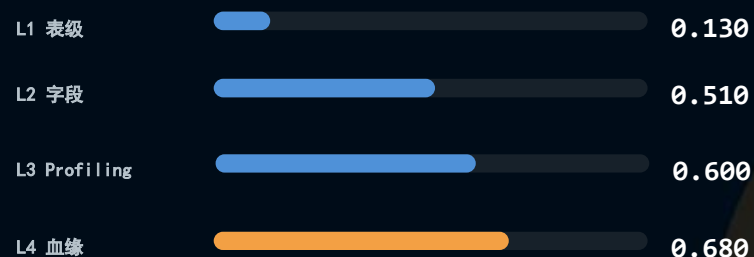
Layer 1 → Layer 4



表召回率提升至 96.7%，复杂问题可继续借助血缘和计算逻辑判断口径。

累计精确率

Layer 1 → Layer 4

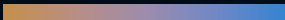


精确率由 0.13 提升至 0.68，分层语义信息持续降低候选噪声。

04

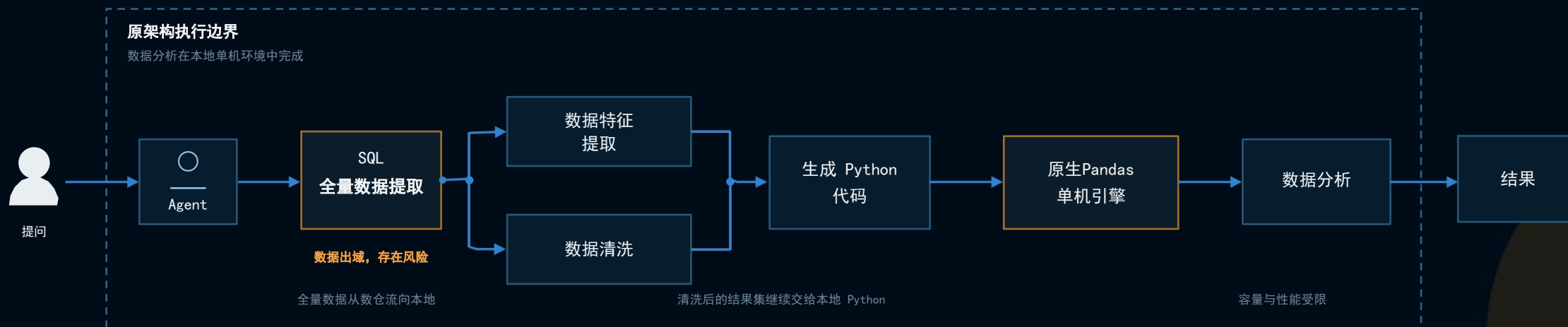
SQL-Python混合计算

🔗 解决“Agent 面对大数据算不动”的问题



通用Agent架构

数据规模通常小于 10GB



本地单机能力有限

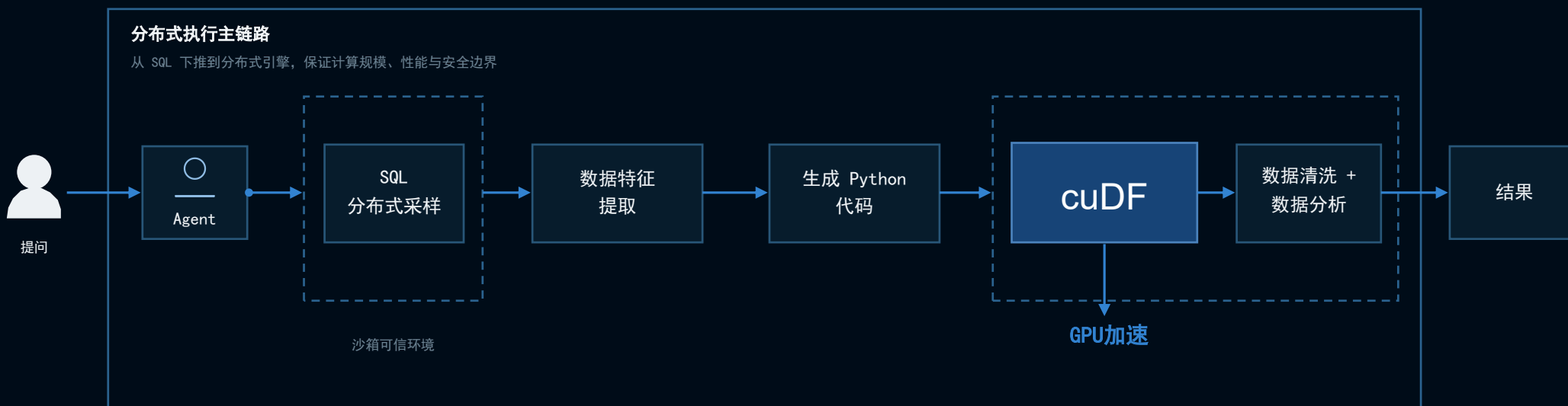
pandas 易受内存墙约束，单机、单并发、CPU 算力限制明显。

数据安全风险高

全量数据下载到本地，增加出域、合规与安全风险。

DataAgent架构：计算规模提升，流程优化

数据规模可达 TB 级别，计算在可信的分布式环境中完成



SQL 负责大规模计算

过滤、聚合、Join、采样下推到数仓或分布式引擎，避免全量数据拉到本地。

Python 负责灵活分析

轻量结果进入可信环境，继续完成特征工程、统计分析、可视化与结果解释。

Agent动态选型

运行时信号



数据规模
分区、预估返回集



计算类型
关系计算 / 灵活分析



安全边界
数据是否留在可信环境



执行反馈
样本、计划与异常状态



Agent
动态判定

D1

生成SQL的时机

01 跨源数据 → SuperSQL 统一取数

✓ 统一数据文件格式

02 试探取数

✓ 试执行

✓ 快速获取表数据缺失信息

D2

执行SQL的程度

01 结果集可被多次分析

✓ 免来回 SQL 取数

02 取数范围可算极限值

✓ 避免后链路分析失败



反馈校正

FEEDBACK LOOP · 模型判断可能偏离边界，需要不停校正

样本不符、计划异常或规模超限时，回到试跑与路由环节，调整 SQL 改写或执行面选择。

直接收益

数据维度	原生 pandas		cuDF（单机）	
	read 耗时	计算耗时	read 耗时	计算耗时
600M	2.04 s	2.91 s	0.08 s	1.19 s
3G	7.91 s	12.77 s	0.14 s	1.82 s
10G	24.43 s	40.43 s	0.79 s	4.27 s
30G	72.31 s	177.77 s	4.35 s	11.64 s

数据不出云

大数据计算在云端或可信环境完成，降低数据下载和本地处理带来的安全风险。

算法不撞墙

避免 pandas 本地内存瓶颈，支持更大规模的数据分析任务，并充分发挥分布式异构算力。

工程不排队

减少分析师等待工程师改写 PySpark 的成本，由 Agent 自动完成 SQL 与 Python 之间的执行分工。

05

总结与展望

总结

Data Agent 的设计实战

围绕数据分析链路，把知识、检索与计算转化为 Agent 可以自主调用的系统能力。

01

懂数据

AI 元数据增强、字段 Profiling、血缘和计算逻辑，让数据资产具备可理解的业务语义。

02

找得准

L1 - L4 渐进式披露和自规划工具调用，按需定位正确表、字段和口径。

03

算得动

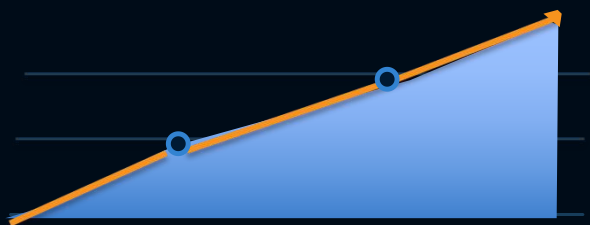
SQL 下推、Python 加速沙箱，让可信环境中的大规模分析可持续执行。

FUTURE ROADMAP

两条主线 持续演进

知识侧 做厚上下文

评测侧 做实长期可信



01

知识侧

广义统一元仓

统一上下文入口，知识从 **单次** → **资产**，覆盖从 **找表** → **全量**

— 知识源覆盖

文档类 iWiki

代码类知识源

Skill

— 知识资产化飞轮

搜表

资源

授权

任务

02

评测侧

常态化评测系统

评测从 **周期性** → **常态化**，评估从 **单点** → **全链路**

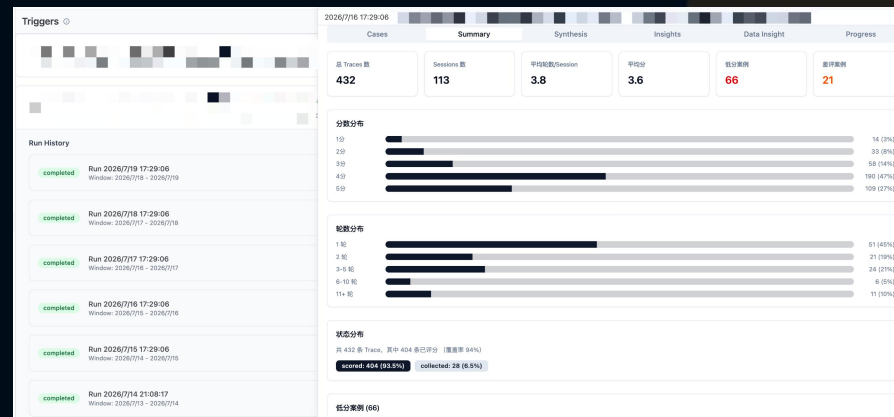
— 持续沉淀评测基准 (GT)

— 联合评估

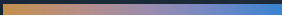
Agent 轨迹

工具调用

成本



THANKS

A short horizontal bar with a gradient from orange to blue, positioned below the word 'THANKS'.